CYBERSECURITY INTERNSHIP

# COMPREHENSIVE FINAL REPORT

Three Tasks —Report and Analysis

| | |
|---|---|
| **Intern Name** | **Alugani Pranay Raj Goud** |
| **Program** | Cybersecurity Internship — Future Interns |
| **Report Date** | March 2026 |
| **Tasks Covered** | Task 1 , Task 2 , Task 3 |
| **Classification** | Internship Submission |

# Table of Contents


**1. Introduction** ..... 3
**2. Website Vulnerability Assessment** ..... 4
2.1 Scope & Approach ..... 4
2.2 Tools Used ..... 4
2.3 Vulnerabilities Identified ..... 4
2.4 Recommendations ..... 5
**3. Phishing Email Detection & Awareness** ..... 6
3.1 Scope & Approach ..... 6
3.2 Email Samples Analysed ..... 6
3.3 Phishing Indicators Found ..... 7
3.4 Awareness & Prevention Guidelines ..... 7
**4. API Security Risk Analysis** ..... 8
4.1 Scope & Approach ..... 8
4.2 Tools Used ..... 8
4.3 Security Risks Identified ..... 9
4.4 Recommendations ..... 10
**5. Consolidated Findings Summary** ..... 10
**6. Skills & Competencies Developed** ..... 11
**7. Conclusion & Reflections** ..... 11

# 1. Introduction

This report documents the work I completed during my Cybersecurity Internship with Future Interns in March 2026. Over the course of the internship, I was given three hands-on security assignments covering three different areas of cybersecurity — website security, email threat analysis, and API security.

The goal of this report is to bring all three of those assignments together into one final document that clearly shows what I did, what I found, and what I learned. Each section covers a different area of work, written in a way that could be understood by both a technical reviewer and a general business audience.

All work was carried out ethically and responsibly. I only tested publicly available platforms that are specifically designed for this kind of security learning. No real systems were attacked, modified, or harmed at any point during this internship.

| | |
|---|---|
| **Intern Name** | Alugani Pranay Raj Goud |
| **Organisation** | Future Interns |
| **Track** | Cybersecurity |
| **Report Date** | March 2026 |
| **Standards Referenced** | OWASP Top 10, OWASP API Security Top 10, NIST CSF |
| **Tools Used** | Chrome DevTools, Browser Analysis, Email Header Analysis, OWASP ZAP (passive) |

# 2. Website Vulnerability Assessment

## 2.1 Scope & Approach

For this part of the internship, I performed a passive security assessment of a public demo website. My job was to look at the website from the outside — the way an attacker or an auditor would — and identify any security weaknesses that could put the site or its users at risk. I did not attempt to hack the site, exploit any vulnerabilities, or interfere with its operation in any way.

| | |
|---|---|
| **Target** | testphp.vulnweb.com (public demo vulnerable site by Acunetix) |
| **Scope** | Public-facing pages only — passive scanning, no exploitation |
| **Testing Type** | Read-only, ethical, non-destructive |
| **Focus** | Identify common web vulnerabilities and misconfigured security controls |

## 2.2 Tools Used

- Browser DevTools — I used this to inspect the HTTP response headers the website sends back, check how cookies are set, and look at the client-side code
- OWASP ZAP in passive scan mode — this tool observes traffic without sending any attacking requests, so it is completely safe to use
- Manual header analysis — I manually compared the headers returned by the site against the recommended security headers from OWASP and Mozilla
- Visual page source inspection — I looked at the visible page source to check whether any sensitive information was being exposed unintentionally

My overall approach followed the OWASP Web Security Testing Guide (WSTG): start by understanding the site, then inspect the headers, identify what is missing or misconfigured, classify the risk level of each issue, and suggest practical fixes.

## 2.3 Vulnerabilities Identified

| # | Vulnerability | Severity | OWASP Ref | Impact |
|---|---|---|---|---|
| 1 | Missing Content-Security-Policy (CSP) header | **High** | A05 - Misconfiguration | Allows XSS attacks |
| 2 | Missing X-Frame-Options header | **High** | A05 - Misconfiguration | Enables clickjacking attacks |
| 3 | Missing X-Content-Type-Options header | **Medium** | A05 - Misconfiguration | MIME-type sniffing risk |
| 4 | Cookies without Secure / HttpOnly flags | **High** | A02 - Cryptographic Failures | Session hijacking risk |
| 5 | No HTTPS enforcement / HSTS missing | **High** | A02 - Cryptographic Failures | Man-in-the-middle attacks |
| 6 | Server version disclosed in headers | **Medium** | A05 - Misconfiguration | Exposes server fingerprint |
| 7 | No Referrer-Policy header | **Low** | A05 - Misconfiguration | Data leakage via referrer |

## 2.4 Recommendations

### Missing Security Headers (Findings 1, 2, 3, 7)

These are some of the easiest fixes available — they require just a few lines of configuration at the server level, but they make a significant difference to the site's security posture. I would recommend adding the following response headers:

- Content-Security-Policy: default-src 'self' — this stops the browser from loading or running any scripts that do not come from the site itself, which blocks the most common form of XSS attack
- X-Frame-Options: DENY — this prevents attackers from loading the site inside a hidden iframe to trick users into clicking on things they did not intend to click (clickjacking)
- X-Content-Type-Options: nosniff — this stops browsers from guessing file types, which can lead to unexpected execution of malicious files
- Referrer-Policy: no-referrer — this stops the browser from sending the previous page URL to third-party sites when a user clicks a link

### Insecure Cookies (Finding 4)

Every session cookie should be set with the Secure flag (so it is only ever sent over HTTPS) and the HttpOnly flag (so JavaScript cannot read it). Without these flags, an attacker who intercepts network traffic or finds an XSS vulnerability could steal a user's session token and take over their account.

### Server Version Disclosure (Finding 6)

The site is currently revealing its exact server software and version number in the response headers. This gives attackers a shortcut — they can look up known vulnerabilities for that exact version and target the site with specific exploits. The fix is simple: remove or suppress the Server and X-Powered-By headers in the server configuration.

# 3. Phishing Email Detection & Awareness

## 3.1 Scope & Approach

For this part of the internship, I worked as a security analyst looking at phishing email samples. Phishing is one of the most common ways that attackers get into organisations — not by breaking through firewalls, but by tricking employees into clicking malicious links or giving away their passwords. My goal was to analyse several phishing email examples, identify what made them dangerous, and put together a practical awareness guide that employees could actually use.

| | |
|---|---|
| **Focus Area** | Phishing email analysis and employee security awareness |
| **Samples Analysed** | 3 phishing email scenarios |
| **Tools Used** | Email Header Analyzer (Google MX Toolbox), Browser-based link inspection |
| **Output** | Phishing indicator report + employee Do's & Don'ts guide |

## 3.2 Email Samples Analysed

### Sample 1 — Fake Account Suspension Email

| | |
|---|---|
| **Subject** | URGENT: Your Account Will Be Locked in 24 Hours |
| **Sender** | security-team@secure-account-verify.com |
| **Malicious Link** | http://secure-account-verify[.]com/login |
| **Classification** | **PHISHING** |
| **Risk Level** | **High** |

### Sample 2 — Fake Bank Alert Email

| | |
|---|---|
| **Subject** | Your Bank Transaction Has Been Flagged |
| **Sender** | alerts@hdfc-bank-secure[.]in |
| **Malicious Link** | http://hdfcbank-secure-login[.]com/verify |
| **Classification** | **PHISHING** |
| **Risk Level** | **High** |

### Sample 3 — Fake IT Department Password Reset

| | |
|---|---|
| **Subject** | Action Required: Reset Your Password Immediately |
| **Sender** | it-support@company-helpdesk[.]net |
| **Malicious Link** | http://reset-password-now[.]xyz/auth |

| | |
|---|---|
| **Classification** | **PHISHING** |
| **Risk Level** | **High** |

## 3.3 Phishing Indicators Found

| # | Indicator | Description | Risk Level |
|---|---|---|---|
| 1 | Spoofed sender domain | **Domain looks similar to legitimate but is fake (e.g. hdfc-bank-secure.in vs hdfcbank.com)** | High |
| 2 | Urgency / fear language | **Phrases like 'Your account will be locked' pressure users to act without thinking** | High |
| 3 | Suspicious URL | **Link domain does not match the claimed organisation; uses HTTP not HTTPS** | High |
| 4 | Generic greeting | **'Dear User' instead of the recipient's actual name — indicates mass phishing** | Medium |
| 5 | Unusual TLD (.xyz, .net) | **Legitimate companies rarely use uncommon domains for official communications** | Medium |
| 6 | No digital signature | **Legitimate corporate emails are typically signed with DKIM/DMARC — absent here** | Medium |
| 7 | Mismatched email header | **Email header 'Reply-To' address differs from the 'From' address** | High |

## 3.4 Awareness & Prevention Guidelines

### Do's — What Employees Should Do

- Always verify the sender's email domain before clicking any link or attachment
- Hover over links to preview the actual URL before clicking
- Report suspicious emails to the IT/security team immediately
- Use multi-factor authentication (MFA) on all accounts to limit damage if credentials are stolen
- Check for HTTPS and a valid certificate before entering any login credentials on a website
- When in doubt, contact the organisation directly through official channels — not via the email

### Don'ts — What Employees Must Avoid

- Never click links in emails that create urgency or fear
- Never download attachments from unknown or unexpected senders
- Never enter your password on a page reached by clicking an email link
- Never share OTPs, passwords, or personal information via email
- Never assume an email is safe because it appears to come from a known brand
- Never reply to suspicious emails — even replies confirm your address is active

# 4. API Security Risk Analysis

## 4.1 Scope & Approach

For this part of the internship, I carried out a security review of a public REST API. APIs are the backbone of modern applications — they power mobile apps, dashboards, and integrations between services. If an API is not properly secured, attackers can access sensitive data, bypass authentication, or delete records without any authorisation. My goal was to test a public API, find any security gaps, and document them the way a security analyst would for a client.

| | |
|---|---|
| **Target API** | JSONPlaceholder — https://jsonplaceholder.typicode.com |
| **API Type** | Public REST API for developer testing |
| **Testing Method** | Read-only analysis using Chrome DevTools (Network tab + Console tab) |
| **Standard** | OWASP API Security Top 10 (2023) |
| **Scope** | GET, POST, DELETE endpoints — no exploitation, no DoS |

## 4.2 Tools Used

- Chrome DevTools — Network Tab: I opened the browser's built-in developer tools and used the Network tab to watch the requests and responses as I visited different API endpoints
- Chrome DevTools — Console Tab: I used the browser console to run simple Fetch API commands to test POST and DELETE requests without needing any external tools
- Manual analysis: I compared each finding against the OWASP API Security Top 10 checklist to classify and rate its severity

Everything was done ethically using JSONPlaceholder, a public API that is specifically built for developer testing and learning. No data was permanently changed, and no real users were affected.

## 4.3 Security Risks Identified

| # | Risk | Severity | Endpoint | OWASP Category |
|---|---|---|---|---|
| 1 | Unauthenticated user data listing | **High** | GET /users | API1 - Broken Object Auth |
| 2 | IDOR — user enumeration by ID | **High** | GET /users/{id} | API1 - Broken Object Auth |
| 3 | Unauthenticated write access (POST) | **High** | POST /posts | API5 - Broken Function Auth |
| 4 | Unauthenticated delete access | **High** | DELETE /posts/1 | API5 - Broken Function Auth |
| 5 | Missing rate limiting on all endpoints | **Medium** | All endpoints | API4 - Resource Consumption |

### Finding Detail — Unauthenticated User Listing (High)

Accessing GET /users in the browser with no token returned a full list of all 10 users including names, emails, phone numbers, and addresses. In a real system this would constitute a serious privacy breach and potential GDPR violation.

### Finding Detail — IDOR / User Enumeration (High)

By incrementing the user ID in the URL (/users/1, /users/2, /users/3...) it was possible to retrieve any individual user's data with no authentication. Requesting /users/999 correctly returned an empty 404 response confirming the ID-based access pattern is exploitable for enumeration.

### Finding Detail — Unauthenticated POST & DELETE (High)

Using the Chrome Console's Fetch API, POST and DELETE requests were sent to the API with no authentication headers. Both returned success responses (201 Created for POST, 200 OK for DELETE), demonstrating that anyone — including anonymous users — can create and destroy records without logging in.

### Finding Detail — Missing Rate Limiting (Medium)

Inspection of all Response Headers showed no X-RateLimit-Limit, X-RateLimit-Remaining, or Retry-After headers on any endpoint. This means there is no throttling in place, leaving the API open to brute force, credential stuffing, and automated scraping.

## 4.4 Recommendations

| Priority | Severity | Recommended Action | Timeline |
|---|---|---|---|
| P1 | Critical | Implement JWT / OAuth2 authentication on ALL endpoints | Immediate |
| P2 | Critical | Add role-based access control — users access only their own data | Immediate |
| P3 | Critical | Restrict POST and DELETE to authenticated and authorised users only | Immediate |
| P4 | High | Implement rate limiting with X-RateLimit-* response headers | Within 1 week |
| P5 | Medium | Replace sequential integer IDs with UUIDs to prevent enumeration | Within 1 month |

## 5. Consolidated Findings Summary

The following table provides a unified view of all security findings identified across the three tasks, giving a complete picture of the cybersecurity risks assessed during this internship.

| # | Area | Finding | Severity | Category |
|---|---|---|---|---|
| 1 | Web Security | **Missing Content-Security-Policy header** | High | Web Security |
| 2 | Web Security | **Missing X-Frame-Options (clickjacking)** | High | Web Security |
| 3 | Web Security | **Cookies without Secure/HttpOnly flags** | High | Web Security |
| 4 | Web Security | **No HTTPS enforcement / HSTS missing** | High | Web Security |
| 5 | Web Security | **Server version disclosed in headers** | Medium | Web Security |
| 6 | Web Security | **Missing X-Content-Type-Options header** | Medium | Web Security |
| 7 | Email Security | **Spoofed sender domain in phishing email** | High | Email Security |
| 8 | Email Security | **Malicious links disguised as legitimate URLs** | High | Email Security |
| 9 | Email Security | **Fear/urgency manipulation tactics** | High | Social Engineering |
| 10 | Email Security | **No DKIM/DMARC email authentication** | Medium | Email Security |
| 11 | API Security | **Unauthenticated access to all user data** | High | API Security |
| 12 | API Security | **IDOR — user enumeration via sequential IDs** | High | API Security |
| 13 | API Security | **Unauthenticated POST (write access)** | High | API Security |
| 14 | API Security | **Unauthenticated DELETE (destructive access)** | High | API Security |
| 15 | API Security | **Missing rate limiting across all endpoints** | Medium | API Security |

## Findings Summary by Severity

| Severity | Count | Tasks Affected |
|---|---|---|
| High | 11 | Web, Email & API Security |
| Medium | 4 | Web, Email & API Security |
| Low | 1 | Web Security |
| Total | 16 | All 3 Tasks |

## 6. Skills & Competencies Developed

Through the three assignments in this internship, I got hands-on experience with a range of cybersecurity skills that I had previously only read about. Here is a summary of what I practised and what I learned:

| Skill Area | What I Did & Learned |
|---|---|
| Website Vulnerability Assessment | Performed a passive audit of a public website and identified 7 security weaknesses using header analysis and OWASP ZAP |
| HTTP Security Header Analysis | Learned how response headers work and why missing ones like CSP and X-Frame-Options create real risk |
| Phishing Email Analysis | Analysed 3 phishing email samples and identified 7 indicators that distinguish a phishing email from a legitimate one |
| Security Awareness Communication | Wrote a clear awareness guide explaining social engineering tactics in plain language for non-technical users |
| API Security Testing | Tested 7 API endpoints using only Chrome DevTools and found 5 significant security issues including unauthenticated access |
| OWASP Framework | Applied both OWASP Top 10 and OWASP API Security Top 10 as the standard framework for classifying all findings |
| Risk Classification | Rated every finding as High, Medium, or Low severity with a clear explanation of why it matters to a business |
| Security Report Writing | Produced structured, professional reports for each area of work that could be shared with a real client or employer |
| Ethical Testing Mindset | All testing was read-only, non-destructive, and stayed within defined scope — no systems were harmed |

## 7. Conclusion & Reflections

This internship gave me a much more grounded understanding of cybersecurity than I expected. Going in, I knew the theory — terms like XSS, phishing, API authentication, and rate limiting — but doing these assignments showed me what it actually looks like when those things go wrong in practice.

The website vulnerability assessment taught me that even simple configuration mistakes — like forgetting to add a security header — can open up serious attack paths. It also showed me that a lot of basic security is not about complex hacking; it is about checking whether the right protections are even in place.

The phishing analysis was eye-opening in a different way. It made me realise how convincing these emails can look, and why so many people fall for them. Writing the awareness guide made me think about how to communicate security risks to non-technical people, which I think is one of the most underrated skills in this field.

The API security analysis was the most technical of the three, but also the most satisfying. Using just a browser and Chrome DevTools, I was able to identify serious security gaps — unauthenticated access, missing rate limiting, and the ability to delete records without any login. It showed me that you do not need expensive tools to do meaningful security work.

Overall, I identified 16 security findings across the three areas — 11 rated High severity and 5 rated Medium or Low. More importantly, I now understand not just what these vulnerabilities are, but why they matter and how to explain them clearly. I am grateful to Future Interns for the opportunity and look forward to building on these skills.

---

**End of Report**

*Alugani Pranay Raj Goud | Cybersecurity | Future Interns | March 2026*